2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS) | 979-8-3503-3071-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICPADS60453.2023.00122

Formalization and Verification of RocketMQ Using CSP

Yiwen Liu, Hongyan Mao*, Ning Qin, Kai Chen
Shanghai Key Laboratory of Trustworthy Computing
East China Normal University, Shanghai, China
hymao@sei.ecnu.edu.cn, {51215902027, 51255902002, 51255902007}@stu.ecnu.edu.cn

***Abstract*—With the development of cloud serverless and microservices architectures, RocketMQ recently has emerged as a compelling pattern of service-to-service communication for them. Nowadays, due to its features of high performance, high reliability, low latency, many companies use the open source version of RocketMQ in their business. The existing works are mainly the applications of RocketMQ to realize the program communications, however there are few formalized models for RocketMQ. It is essential to model and verify the RocketMQ system abstracted from its architecture in a formal method to ensure the messaging reliability. In this paper, we focus on the data messaging in production and consumption mechanism to facilitate the overall modeling. Six communication components are extracted including Producer, Consumer, Master Broker, Slave Broker, NameServer, and Clock for describing the temporal process. Then we formalize and model the RocketMQ system using Communicating Sequential Processes (CSP) to describe communicating process among those components. Furthermore, we implement the model of RocketMQ and verify six properties, such as Deadlock Freedom, Consistency, Parallelism, Fault Tolerance, Sequentiality and HeartBeat Mechanism based on the model checker Process Analysis Toolkit (PAT). Finally, our verification results show that the model can satisfy these properties, indicating its messaging reliability can be guaranteed.**

***Index Terms*—RocketMQ, CSP, Formaization, HeartBeat Mechanism.**

I. Introduction

With the rapid growth of big data and Internet business, a large amount of data is being produced, transmitted and consumed continuously. The various applications need to deal with more and more tasks for high concurrent access and massive data processing in actual scenarios. To meet the demands of massive data processing, the primary goals of system development are high availability, scalability, high reliability, and quick data response. Nowadays many Internet companies adopt stream computing, which depends on fast and stable transmission channels to realize the stream data processing and data transmission. The distributed message queue systems have become increasingly popular in streaming data processing applications [1] [2].

A distributed message queue system is a significant component for message communication between applications. The messaging is based on reliable message queues in the distributed messaging system, and there are two main messaging modes: the point-to-point (PTP) [3] model and the publish-subscribe (Pub-Sub) [4] model. In the traditional point-to-point messaging systems, such as ActiveMQ [5], ZeroMQ [6] and RabbitMQ [7], a producer produces a message and sends it to a queue. The consumers pull and consume data from the queue sequentially. Once a message is consumed, it is removed from the message queue. Unlike the point-to-point, in a publish-subscribe messaging system, the messages sent by producers are divided into different topics based on their contexts. Meanwhile, the consumer can subscribe to multiple topics and consumer data in those topics. Besides, data under the same topic can be consumed by different consumer groups, and the consumed data will not be deleted immediately. Apache RocketMQ [8] is an excellent representative of distributed message queuing system.

Apache RocketMQ, a distributed messaging middleware developed by Alibaba, was the first designed for message handling on a trillion scale and has the characteristics of high throughput, low latency, massive accumulation. Nowadays, the open source version of RocketMQ has been widely used in their business by many Internet companies, such as Alibaba, Huawei, and iFLYTEK, etc. In addition, many studies have recently turned to design their service frameworks and applications integrating RocketMQ for high performance purposes. Ma et al. [9] proposed a MQTT protocol message push server based on RocketMQ, which overcomes many shortcomings of the MQTT server in the distributed deployment. Yang et al. [10] proposed a blockchain service in data collection and analysis and RocketMQ is used for improving the efficiency of the process of writing data into the blockchain and querying the blockchain. Although considerable research has been devoted to applying RocketMQ to realize the communication between applications, rather less attention has been paid to the interaction behavior of components in the RocketMQ system. With its widespread use, it is necessary to analyze its model abstracted from the architecture and to verify the reliability of RocketMQ. It is of great significance for developers to adopt a formal method to verify the reliability of software architecture and improve its design.

In this paper, six communication components are abstracted including Producer, Consumer, Master Broker, Slave Broker, NameServer. And the RocketMQ is modelled using Communicating Sequential Processes (CSP) [11]. We analyze the constructed model and verify six properties including Deadlock Freedom, Consistency, Parallelism, Sequentiality, HeartBeat Mechanism and Fault Tolerance in a model checker named Process Analysis Toolkit (PAT) [13].

In detail, the rest of this paper is organized as follows:

2690-5965/23/$31.00 ©2023 IEEE
DOI 10.1109/ICPADS60453.2023.00122

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:17:11 UTC from IEEE Xplore. Restrictions apply.

the overview of the RocketMQ system and process algebra CSP are introduced in Section II. In Section III, we formalize the model of RocketMQ using CSP to describe the behaviors between components. In the next section, we implement the model and verify the six properties in PAT. Finally, we give a conclusion and discussion on future work in Section V.

II. Related Work

In this section, the overview architecture of RocketMQ is illustrated including its communication components, interaction behaviors as well as basic terminology. Besides, we give an introduction to the main concept and syntax of CSP.

A. RocketMQ

The architecture of RocketMQ is shown in Fig.1. We have seen the main communications of RocketMQ architecture. Firstly, the basic terminology in RocketMQ is as follows:

- **Producer**: It is also known as a message publisher. The producer is responsible for producing a certain topic message and sending it to a broker selected by routing information of its local cached topic router table. And the producer pulls the routing information from the nameserver every default time interval.
- **Broker**: It is responsible for receiving messages sent from producers and storing them. The broker server is divided into master broker and slave broker. A master broker can have multiple slave brokers correspondingly. Once a master broker hangs up, the consumer can consume data from its slave broker.
- **NameServer**: It is a topic routing registry center providing routing service. The producer and consumer can know the routing information of the entire Broker cluster through the nameserver, so as to deliver and consume the message. The nameserver maintains the contact of producers and consumers by heartbeat packets.
- **Topic**: The topic refers to a collection of one kind of message sent by producers. Each topic contains several messages and one message can only belong to one topic.
- **Message Queue**: The message queue is one of the shards of a topic on a broker, called a topic partition. It is the basic unit of resource allocation for load balancing.
- **Consumer**: It is responsible for receiving and consuming messages from topics. If consumers do broadcast consumption, each consumer consumes all data partitions in a topic. While under the clustering mode, each consumer consumes part of the message queues of a topic averagely. A consumer group has multiple consumers with the same consumption behavior.

In the next, we introduce the message transmission of RocketMQ. The broker server registers its own routing information with the nameserver. The producers and the consumers establish a connection with the nameserver and can obtain topic routing information from the nameserver. The producer produces a message of a certain topic, obtains the routing information of the topic from the nameserver and

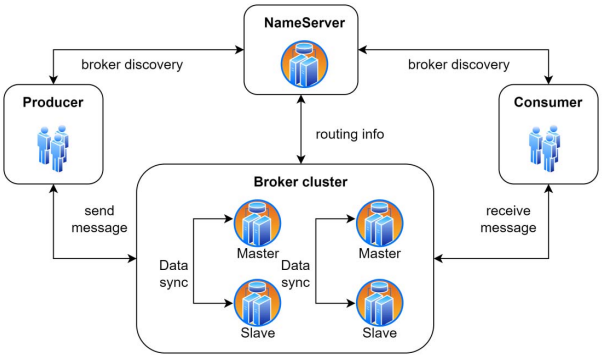


Fig. 1. The architecture of RocketMQ.

sends the message to a broker by routing information. The broker stores message in the topic and its corresponding slave brokers synchronizes information. The consumer connects to the nameserver and consumes data according to the queue allocated by the broker.

B. CSP

Communicating Sequential Processes (CSP) was proposed by C.A.R Hoare [11], which is a classic formal language to describe the communication behaviors in concurrent systems. And CSP has been successfully applied in many fields, such as networks protocols [14] [15] [16] and distributed systems [13]. We give the following syntax of the CSP language used to describe the process, where P and Q are processes, a denotes the event and c represents a channel.

$$P, Q ::= Skip|Stop|a \rightarrow P|c?x \rightarrow P|c!v \rightarrow P|P\Box Q|P\|Q|P;Q.$$

- $Skip$ represents a process terminates successfully and does not run into a deadlock state.
- $Stop$ denotes that a process run into a deadlock state.
- $a \rightarrow P$ is a prefix, which executes the event a and then behaves like process P.
- $c?x \rightarrow P$ P receives a message by channel c, stores the value in the variable x, and then behaves like process P.
- $c!v \rightarrow P$ P sends message x by channel c and then acts like process P.
- $P\Box Q$ represents the general choice of either P or Q, which is decided by the environment.
- $P\|Q$ denotes process P and process Q execute concurrently and they must synchronize with the same events.
- $P;Q$ represents process P and Q execute in sequence.

III. Modelling RocketMQ

In this section, we abstract and model the architecture to describe its communicating processes. Due to the complex structure of the RocketMQ, which integrates many modules, we focus on data messaging in production and consumption to facilitate the overall modeling in this paper. We model the RocketMQ system using CSP, a classic formal language especially in describing concurrent systems. In addition to the five core components of RocketMQ, we define a Clock component to better describe the temporal process in RocketMQ.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:17:11 UTC from IEEE Xplore. Restrictions apply.

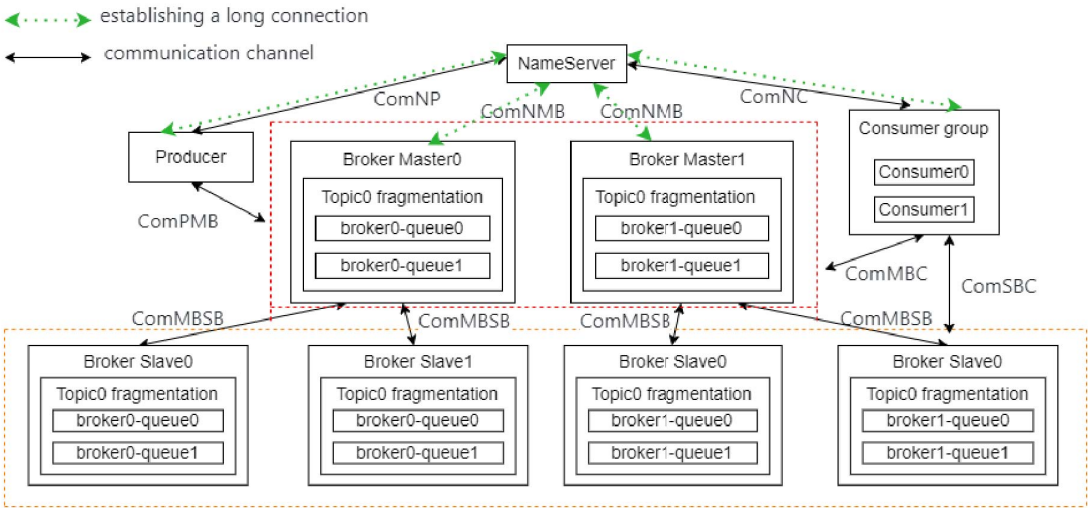


Fig. 2. Communications using channels among components.

A. Overall Modeling of RocketMQ

The data transmission process between the components of the overall modeling can be seen in Fig.2. The black lines represent data transmit channels among components and the green dashed lines represent channels for establishing a long connection among components. There are six crucial modules running in parallel through their own corresponding channels, including Producer, Consumer, Master Broker, Slave Broker, NameServer and Clock. The Clock process is used to describe global time. The whole system is modeled as below.

$$
\begin{aligned}
&System() =_{df} \\
&\quad |||pid \in PID, cid \in CID, mbid \in MBID, sbid \in SBID \\
&\quad (NameServer() \,[|COM_PATH|]\, Producer_{pid}() \\
&\quad [|COM_PATH|] MasterBroker_{mbid}() \\
&\quad [|COM_PATH|] SlaveBroker_{sbid}() \\
&\quad [|COM_PATH|] Consumer_{cid}() \\
&\quad [|COM_PATH|] Clock())
\end{aligned}
$$

where the symbol pid represents a producer's ID, which is the unique identification of each producer. And PID is the range of pid, and other symbol, such as $mbid$, $sbid$, cid, $MBID, SBID, CID$, have similar meanings.

According to the RocketMQ message transmission mechanism, using COM_PATH can represent channels among components shown in Fig.2, which contains ComNP, ComNC, ComNMB, ComPMB, ComMBSB, ComMBC, ComSBC and Time. The explanations of these channels are listed in TableI.

B. NameServer

In RocketMQ, nameserver is a routing registry center and stores topic registry information. The broker sends a heartbeat packet to the nameserver periodically to inform itself still alive. Nameserver checks the time since the last heartbeat packet was received at a regular interval. Each producer and consumer establishes a connection with nameserver. Nameserver provides a list of the certain topic for them. The behavior of $NameServer()$ is modelled as below.

TABLE I
THE CHANNELS OF THE MODEL

Channels	Description
ComNP	Transmitting data and routing packet between producer and nameserver
ComNC	Transmitting data and routing packet between consumer and nameserver
ComNMB	Transmitting data and heartbeat packet between master broker and nameserver
ComPMB	Transmitting data between producer and master broker
ComMBSB	Transmitting data between master broker and slave broker
ComMBC	Transmitting data between master broker and consumer
ComSBC	Transmitting data between slave broker and consumer
Time	Transmitting current global time between clock and other process

$$
\begin{aligned}
&NameServer() =_{df} \\
&(|||mbid : \{0..MB-1\}@NSHeart_{mbid}())|||NSPro() \\
&|||(|||cid : \{0..C-1\} NSCon_{cid}()) \\
&|||(|||mbid : \{0..MB-1\}@Check_{mbid}());
\end{aligned}
$$

For the process $NSHeart()$, it receives heartbeat packet from borker and records brokers' state in local router information. The process $NSPro()$ and $NSCon()$ represent that the $NameServer$ sends routing information to producer and consumers respectively.

$$
\begin{aligned}
&NSheart_{mbid}() =_{df} (initial \rightarrow NSHeart_{mbid}()) \\
&\quad \square(ComPMB_{mbid}?heartBeat \\
&\quad \{brokerAlive[mbid] = 1\} \rightarrow NSHeart_{mbid}()); \\
&NSPro() =_{df} ComNP?request \rightarrow \\
&\quad ComNP!brokerAlive \rightarrow NSPro(); \\
&NSCon_{cid}() =_{df} ComNC_{cid}?request \rightarrow \\
&\quad ComNC_{cid}!brokerAlive \rightarrow NSCon(cid)
\end{aligned}
$$

For the process $Check()$, nameserver provides a heartbeat mechanism to check whether the broker is still alive. Nameserver will save the entire routing information about the Broker cluster and the queue information for the consumers query. Nameserver checks the last time of it received a

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:17:11 UTC from IEEE Xplore. Restrictions apply.

heartbeat packet at regular intervals nT. Upon checking, once the interval is exceeded, it considers the broker to be offline, and removes the broker from the loucl routing information $brokerAlive$.

$$Check_{mbid}() =_{df} Time!Request \rightarrow Time?start \rightarrow \left(\begin{array}{l} \left(\begin{array}{l} \left(\begin{array}{l} break\{brokerAlive[mbid] = 0;\} \rightarrow \\ updatecheck\{ntime = start;\} \rightarrow \\ Check_{mbid}() \end{array} \right) \\ \lhd\ start - MBtime[mbid] > nmbT \rhd \\ updatecheck\{ntime = start;\} \rightarrow Check_{mbid}() \end{array} \right) \\ \lhd\ start - ntime > nT \rhd \\ Check_{mbid}() \end{array} \right)$$

C. Producer

The producer is mainly responsible for producing and sending messages to broker. Each producer has a unique ID marked as pid. In addition, a producer sends a message of a topic to a certain broker get from nameserver. We formalize producer process as below.

$$Producer_{pid}() =_{df} ProSend()|||ProConnect()$$

For process $ProSend()$ is responsible for producing messages. It sends messages generated by the systems to brokers. RocketMQ provides multiple paradigms of sending, and synchronous sending is adopted which requires the confirmation information return from the Broker in this paper. Producer selects the corresponding Broker cluster queue for message delivery through message queue's load balancing module.

$$\begin{array}{l} ProSend() =_{df} send.Topic.curMbid.curMqid\{ \\ \quad AllocateAverage()\} \rightarrow \\ \quad ComPMB[cur_mbid]!Topic.cur_mqid.Data \rightarrow \\ \quad ComPMB[cur_mbid]?p_ack \rightarrow \\ \quad intoMessageQueue(i); ProSend(); \end{array}$$

The process $ProConnect()$ means Producer regularly pulls information from nameserver. Then the Producer can know the routing information of the entire Broker cluster through the nameserver, so as to deliver and consume the message.

$$\begin{array}{l} ProConnect() =_{df} Time!Request \rightarrow Time?start \rightarrow \\ \quad \left(\begin{array}{l} ComNP!Request \rightarrow ComNP?brokerLiveInfo \\ \{localP = brokerLiveInfo; ptime = start;\} \\ \lhd\ start - ptime > 10 \rhd \\ ProConnect() \end{array} \right) \end{array}$$

D. Broker Server

Broker Server is responsible for the storage, delivery and query of messages and guarantee high availability. The Broker is divided into the Master and the Slave. One Master can correspond to multiple Slaves. However, one Slave can only correspond to one Master. The Master can also deploy multiple. The master broker process and the slave server are modeled respectively below.

For process master broker, there are two communication processes: the master broker writes the messages published by the producer, sends feedback information to the producer and synchronizes the information from the slave server; forwarding consumption specific messages consumed by the consumer. The overall communication process of the master broker is as follows:

$$\begin{array}{l} MasterBroker_{mbid}() =_{df} BrokerHeart_{mbid}()||| \\ \quad (|||sbid : \{0..SB-1\}MasterBrokerPub_{mbid,sbid}())||| \\ \quad (|||cid : \{0..C-1\}@MasterBrokerSub_{mbid,cid}()); \end{array}$$

The process $BrokerHeart$ works in heartbeat mechanism. The Broker starts, keeps a long connection with nameserver, and sends heartbeat packets periodically. After the connection is successful, there is a mapping relationship between Topic and Broker in the nameserver. In the formal process of $BrokerHeart$, Borker asks Clock for the current time firstly. If the time interval is less than heartbeat intervalbT, the broker sends a request to Clock again; otherwise, Provider sends heartbeat packet to nameserver directly and this process cycles continuously.

$$BrokerHeart_{mbid}() =_{df} Time!Request \rightarrow Time?t \rightarrow \left(\begin{array}{l} \left(\begin{array}{l} ComPMB[mbid]!HeartBeat \\ \{MBtime[mbid] = t;\} \rightarrow \\ BrokerHeart_{mbid}() \end{array} \right) \\ \lhd\ t - MBtime[mbid] > bT \rhd BrokerHeart_{mbid}() \end{array} \right)$$

$$\begin{array}{l} MasterBrokerPub_{mbid,sbid}() =_{df} \\ \quad ComPMB[mbid]?topic.mqid.data \rightarrow \\ \quad ComPMB[mbid]!Ack \rightarrow \\ \quad ComMBSB[mbid * SB + sbid]?req \rightarrow \\ \quad ComMBSB[mbid * SB + sbid]!Data \rightarrow \\ \quad ComMBSB[mbid * SB + sbid]?ack \rightarrow \\ \quad MasterBrokerPub_{mbid,sbid}(); \end{array}$$

$$\begin{array}{l} MasterBrokerSub_{mbid,cid}() =_{df} \\ \quad ComMBC[mbid * C + cid]?request \rightarrow \\ \quad ComMBC[mbid * C + cid]!Data \rightarrow \\ \quad MasterBrokerSub(mbid, cid); \end{array}$$

For the slave broker process, when the master broker is down, its corresponding slave broker can provide services to consumers. The master-slave structure of RocketMQ plays an important role in ensuring data reliability. The slave server process synchronizes the data of the master server through the channel ComMBSB, and provides services to the consumers through the channel ComSBC when the master server fails, that is, the consumers can still pull the data they need through the channel ComSBC. The slave server process is specifically defined as follows:

$$\begin{array}{l} SlaveBroker_{mbid,sbid}() =_{df} SlaveBrokerSync_{mbid,sbid}() \\ \quad |||(|||cid : 0..C-1@SlaveBrokerSub_{mbid,sbid,cid}()) \end{array}$$

$$\begin{array}{l} SlaveBrokerSync_{mbid,sbid}() =_{df} \\ \quad ComMBSB[mbid * SB + sbid]!Request \rightarrow \\ \quad ComMBSB[mbid * SB + sbid]?data \\ \quad \{dataSycn[mbid][sbid] = 1;\} \rightarrow \\ \quad ComMBSB[mbid * SB + sbid]!Ack \rightarrow \\ \quad SlaveBrokerSync_{mbid,sbid}(); \end{array}$$

$$\begin{array}{l} SlaveBrokerSub_{mbid,sbid,cid}() =_{df} \\ ComSBC[mbid * SB * C + sbid * C + cid]?request \rightarrow \\ ComSBC[mbid * SB * C + sbid * C + cid]!Data \rightarrow \\ SlaveBrokerSub_{mbid,sbid,cid}; \end{array}$$

E. Consumer

In RocketMQ transmission mechanism, the consumption module consists of multiple consumer groups, and each consumer group consists of several consumers with the same

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:17:11 UTC from IEEE Xplore. Restrictions apply.

consumption behavior and logic. After getting routing information from nameserver, the consumers in the same consumer group consume the information of the specified topic partitions respectively according to load balancing algorithm (i.e., AllocateAveragely in this paper). The behavior process of the consumer can be divided into two parts. If the topic requested by the consumer is legal, the message can be pulled from the master broker through the channel ComMBC. When the master broker crashes, the consumer pulls data from its corresponding slave broker through the channel ComSC. If the consumer request is illegal, the consumer request is ignored.

$$Consumer_{cid}() =_{df} ConConnect_{cid}()|||$$
$$(ConsumerToMB_{cid}()\Box ConsumerToSB_{cid}())$$

The process ConConnect() is same as the process $ProConnect()$, which represents Consumers regularly pull routing information from nameserver, so as to consume the message. The process $ConsumerToMB$ and $ConsumerToSB$ represent the consumption process of consumption from the master brokers and consumption from the slave brokers respectively.

$$ConConnect_{cid}() =_{df} Time!Request \rightarrow Time?start \rightarrow$$
$$\left(\begin{array}{l} ComNC[cid]!Request \rightarrow \\ ComNC[cid]?broker_live_info \\ \{localC[cid*MB] = broker_live[0]; \\ localC[cid*MB+1] = broker_live[1]; \\ ctime[cid] = start;\} \rightarrow \\ ConConnect() \\ \lhd\ start - ctime[cid] >= cT \rhd \\ ConConnect(); \end{array}\right)$$

$$ConsumerToMB_{cid}() =_{df}$$
$$subscribe.Request.c_mbidc_mbid = cid; \rightarrow$$
$$ComMBC[c_mbid*C+cid]!Request \rightarrow$$
$$ComMBC[c_mbid*C+cid]?data$$
$$\{DataConsume[cid][0][cid] = 1;$$
$$DataConsume[cid][1][cid] = 1;\} \rightarrow$$
$$ConsumeMQ(cid); ConsumerToMB_{cid}();$$

$$ConsumerToSB_{cid}() =_{df}$$
$$subscribe.Request.c_mbidc_mbid = cid; \rightarrow$$
$$ComSBC[c_mbid*SB*C+sbid*C+cid]!Request$$
$$\rightarrow ComSBC[c_mbid*SB*C+sbid*C+cid]?data$$
$$\{DataSlave[c_mbid][sbid][0][cid] = 1;$$
$$DataSlave[c_mbid][sbid][1][cid] = 1;\} \rightarrow$$
$$ConsumerToSB_{cid}();$$

F. Clock

We abstract Clock process to better describe the process of RocketMQ and it is used to represent the global clock. Once other processes ask $Clock$ for the time via the channel $Time$, $Clock$ will send back the current time t which is a positive integer. The processes of $Clock(t)$ can be described as follows.

$$Clock(t) =_{df} (tick \rightarrow Clock(t+1))$$
$$\Box(Time?request \rightarrow Time!t \rightarrow Clock(t));$$

IV. Verification

In this section, we implement RocketMQ model mentioned in SectionIII. To ensure the reliability of the model, the properties verification of our system are carried out with PAT. And the results of properties verification are shown at the end.

A. Implementation in PAT

In this subsection, we define a set of constants to represent the number of each component in the model. P, T, C, MB, SB, MQ denote the number of the producers, the topic, the consumers, the master brokers, the slave brokers and message queue. The model checking tool PAT's state search is to explore all possible state, though in order to avoid the problem of state explosion, we randomly set P, T, C, MB, SB, MQ to be $1, 1, 2, 2, 2, 2$, respectively. To verify that these processes execute correctly, we also define some arrays to determine the state of the data in transmission. Some of the declarations are given as below:

$$varMB_{ack}[MB] = [0, 0];$$
$$varSB_{ack}[SB] = [0, 0];$$
$$varstateSlave[MB][SB] = [0, 0, 0, 0];$$
$$varDataMaster[MB][MQ][C] = [0, 0, 0, 0, 0, 0, 0, 0];$$
$$varDataSlave[MB][SB][MQ][C] =$$
$$[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0];$$

The $MB_{ack}[mbid]$ represents whether the data of the $mbid-th$ master broker is synchronized successfully or not. $SB_{ack}[sbid]$ represents whether the $sbid-th$ slave broker synchronizes data from its master broker. $stateSlave$ is the message written state of slave broker. The $DataMaster$ and $DataSlave$ indicate consume records consumed by consumers from master brokers and slave brokers respectively. The initial values of these array variables are set to 0, where the 1/0 value represent true/false respectively.

Property 1: Deadlock Freedom

The model should not run into a deadlock state, which is the basis for model running. In PAT, the following assertion is used for describing this property.

$$\#assert\ System()\ deadlockfree;$$

The Deadlock Freedom property is used to verify whether our system is in the deadlock state or not.

Property 2: Consistency

In message queue, data consistency means that if a message can be consumed by the Consumer, then even if the master broker on which it resides goes down, it can still be read on the its corresponding slave broker. In RocketMQ architecture, the slave brokers synchronize their corresponding master broker to avoid losing data. When the master broker break down, its corresponding slave brokers provide consume service. The definition of Consistency and the assertion are defined as below.

$$\#define\ Consistency(MB_{ack}[0] == 1 \&\& MB_{ack}[1] == 1);$$
$$\#assert\ System()\ reaches\ Consistency;$$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:17:11 UTC from IEEE Xplore. Restrictions apply.

We define a varible Consistency, where $MB_{ack}[mbid]$ means that whether the $mbid-th$ broker completes master-slave synchronization.

Property 3: HeartBeat Mechanism

The heartbeat mechanism of RocketMQ is designed to detect the running status of the brokers, maintains a heartbeat status table of the brokers, updates the information of the routing table by heartbeat packet. When the time interval exceeds max time interval, the nameserver checks the heartbeat of the broker. The definition of HeartBeat Mechanism is described as follows:

$$
\begin{aligned}
&\#define\ beat1\\
&\quad((clockTime - MBtime[0]) >= 12 \&\& brokerAlive[0] == 0)\\
&\quad\|((clockTime - MBtime[1]) >= 12 \&\& brokerAlive[1] == 0);\\
&\#define\ beat2\\
&\quad((clockTime - MBtime[0]) < 12 \&\& brokerAlive[0] == 1)\\
&\quad\|((clockTime - MBtime[1]) < 12 \&\& brokerAlive[1] == 1);\\
&\#define\ HeartBeatMechanism(beat1\|beat2);\\
&\#assert\ System()\ reaches\ HeartBeatMechanism;
\end{aligned}
$$

In this paper, we set the time interval for detecting whether broker heartbeat packet become invalid to 12 seconds. The $clockTime$ is the time when nameserver currently receives the heartbeat packet, and $MBtime[mbid]$ is the time when nameserver last received the $mbid{-}th$ master broker heartbeat packet. The $brokerAlive$ represents the running status of the brokers. When $brokerAlive[mbid] = 1$, it means that nameserver checks that $mbid - th$ master broker is running without trouble, and when $brokerAlive[mbid] = 0$, it means that $mbid{-}th$ master broker is down. The assertion describes that the model satisfies that once the time interval between the current time by nameserver received and the last time it received a broker's heartbeat packet is greater than 12, the broker become unavailable(i.e., $brokerAlive[mbid] = 0$). By the same token, if it is less than 12, the broker is available.

Property 4: Parallelism

Parallel consumption can improve the total consumption throughput in actual application scenarios. In this paper, we apply Parallelism to describe the Parallel consumption reliability of RocketMQ. Parallelism means that the system allows multiple consumers subscribe services concurrently, the processes do not interfere with each other. The assertion is defined as below:

$$
\begin{aligned}
&\#define\ Parallelism\\
&\quad(DataMaster[0][0][0] == 1\\
&\quad\&\&DataMaster[0][1][1] == 1\\
&\quad\&\&DataMaster[1][0][0] == 1\\
&\quad\&\&DataMaster[1][1][1] == 1);\\
&\#assert\ System()\ reaches\ Parallelism;
\end{aligned}
$$

Property 5: Fault Tolerance

The fault tolerance of RocketMQ is achieved by master-slave synchronous replication. In the RocketMQ system, when a master broker breaks down, its corresponding slave brokers provide services, so that consumers can still consume correct data. The entire system does not crash, and processes can still run and transmit messages normally. This property is defined here as FaultTolerance. The assertion about this property is defined as below:

$$
\begin{aligned}
&\#define\ P1\\
&\quad(DataSlave[0][0][0][0] == 1\\
&\quad\|DataSlave[0][1][0][0] == 1)\\
&\#define\ P2\\
&\quad(DataSlave[0][0][1][1] == 1\\
&\quad\|DataSlave[0][1][1][1] == 1\\
&\#define\ P3\\
&\quad(DataSlave[1][0][0][0] == 1\\
&\quad\|DataSlave[1][1][0][0] == 1)\\
&\#define\ P4\\
&\quad(DataSlave[1][0][1][1] == 1\\
&\quad\|DataSlave[1][1][1][1] == 1)\\
&\#define\ FaultTolerance(P1\&\&P2\&\&P3\&\&P4);\\
&\#assert\ System()\ reaches\ FaultTolerance;
\end{aligned}
$$

Property 6: Sequentiality

Sequentiality of RocketMQ means that the order in which the messages are consumed by the consumer and those produced by the producer is the same. The producer generates messages in ascending order, so the consumer always consumes messages in ascending order as well.

$$
\begin{aligned}
&\#define\ mqueue0\\
&\quad(c0mq0.Count() == 0\|c0mq0.Count() == 1\\
&\quad\|(c0mq0.Count() == 2\&\&\\
&\quad c0mq0.First() < c0mq0.Last()))\\
&\#define\ mqueue1\\
&\quad(c0mq1.Count() == 0\|c0mq1.Count() == 1\\
&\quad\|(c0mq1.Count() == 2\&\&\\
&\quad c0mq1.First() < c0mq1.Last()))\\
&\#define\ mqueue2\\
&\quad(c1mq0.Count() == 0\|c1mq0.Count() == 1\\
&\quad\|(c1mq0.Count() == 2\&\&\\
&\quad c1mq0.First() < c1mq0.Last()))\\
&\#define\ mqueue3\\
&\quad(c1mq1.Count() == 0\|c1mq1.Count() == 1\\
&\quad\|(c1mq1.Count() == 2\&\&\\
&\quad c1mq1.First() < c1mq1.Last()))\\
&\#define\ Sequentiality(mqueue0\&\&mqueue1\\
&\quad\&\&mqueue2\&\&mqueue3);\\
&\#assert\ System()\ reaches\ Sequentiality;
\end{aligned}
$$

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:17:11 UTC from IEEE Xplore. Restrictions apply.

B. Verification Results

We implement the model in PAT and its verification results are showed in Fig.3. The verification results shows that the six properties are all valid. The constructed model does not run into a deadlock state and satisfy reliability properties.

- The property Deadlock Freedom means that the formalization model we constructed does not run into a deadlock state.
- The property Consistency is the basis of the messaging reliability and indicates that each slave broker synchronizes and stores the data from its corresponding master broker.
- The property HeartBeat Mechanism is designed to check whether the component maintains a heartbeat. When the time interval exceeds max time interval, the nameserver checks the heartbeat of the broker to ensure connection.
- The property Parallelism represents that each consumer in the same consumer group can consume data of its corresponding topic partitions without interference.
- The property Fault Tolerance ensures the messaging reliability of the RocketMQ system. When a master broker breaks down, its corresponding slave brokers provide subscription service and the system does not crash.
- The property Sequentiality means that the order in which the messages are consumed by the consumer and those produced by the producer is the same.

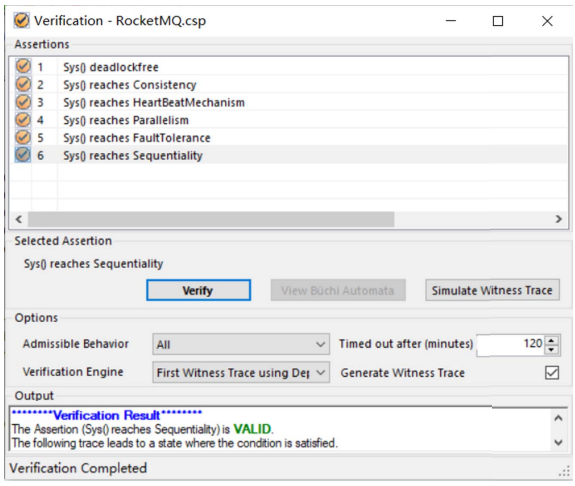


Fig. 3. Verification Results in System().

V. Conclusion

In this paper, we proposed a formal model of the RocketMQ system using CSP in PAT. The communication components are presented and described as processes respectively, including nameserver, producer, consumer, master broker and slave broker. Moreover, Six properties are defined to verify the correctness of model and validated by the model checker PATautomatically. The verification results demonstrate that our model satisfies these transmission properties and the reliability of RocketMQ can be guaranteed. The formal verification of RocketMQ message system is still a challenge due to its complexity architecture. In the future, we would refine our model and detect other potential properties that developers are widely concern about, such as the security aspects.

Acknowledgment

This work is supported by the National Key Research and Development Program of China (No.2020YFB2104202, No.2022YFB3305102), the National Natural Science Foundation of China (No.61872145, No.62032034), the Key Laboratory of Embedded System and Service Computing Ministry of Education (Tongji University) (No.ESSCKF2021-03), and the "Digital Silk Road" Shanghai International Joint Lab of Trustworthy Intelligent Software (No.22510750100).

References

[1] S. N. Raje, "Performance Comparison of Message Queue Methods," University of Nevada, Las Vegas, 2019.
[2] Y. Jiang, Q. Liu, C. Qin, J. Su, and Q. Liu, "Message-oriented middleware: a review," in 2019 5th International Conference on Big Data Computing and Communications (BIGCOM), 2019: IEEE, pp. 88-97.
[3] M. Consultant, "Point-to-Point (PTP) and Point-to-Multipoint (PMP) Wireless Systems and Antennas". Wireless Access Networks, 2001.
[4] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," ACM computing surveys (CSUR), vol. 35, no. 2, pp. 114-131, 2003.
[5] B. Snyder, D. Bosanac, and R. Davies, "Introduction to apache activemq," Active MQ in action, pp. 6-16, 2017.
[6] ZeroMQ: the guide. [Online]. Available: http://zeromq.org.
[7] RabbitMQ. [Online]. Available: https://www.rabbitmq.com/tutorials /amqp-concepts.html.
[8] RocketMQ architecture. [Online]. Available: https://rocketmq.apache. org.
[9] Y. Ma, R. Yan, J. Sun and K. Yao, "A MQTT protocol message push server based on RocketMQ," in 2017 10th International Conference on Intelligent Computation Technology and Automation (ICICTA), 2017: IEEE, pp. 295-298.
[10] J. Yang, Z. Lu, R. Xu, J. Wu, X. Xuan, and J. Cheng, "DCAB: An Effective Data Collection and Analysis Service for Blockchain," in 2021 IEEE International Conference on Services Computing (SCC), 2021: IEEE, pp. 99-106.
[11] C. A. R. Hoare, "Communicating sequential processes," Communications of the ACM, vol. 21, no. 8, pp. 666-677, 1978.
[12] PAT: Process Analysis Toolkit. In: [Online] Available: http://pat.comp.nus.edu.sg/.
[13] Z. Hou, J. Yin, and H. Zhu, "Formalization and Verification of Dubbo Using CSP," SEKE, 2021: 1-6.
[14] S. N. Andersen, A. E. Espe, S. Hendseth, and G. Mathisen, "Formalising Nondeterministic Communication in Wireless Sensor Networks Using CSP," in 2021 10th Mediterranean Conference on Embedded Computing (MECO), 2021: IEEE, pp. 1-4.
[15] J. Jiang, H. Mao, R. Shao, and Y. Xu, "Formal verification and improvement of the PKMv3 protocol using CSP," in 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), 2018, vol. 2: IEEE, pp. 682-687.
[16] N. Qin, H. Mao, Y. Liu and K. Chen, "Modeling and Verifying Privacy-Preserving Authentication Scheme for VANET Using CSP," 2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC), Torino, Italy, 2023, pp. 747-752, doi: 10.1109/COMPSAC57700.2023.00103.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:17:11 UTC from IEEE Xplore. Restrictions apply.